STRATADB

A Practical Textbook on Building a Storage Engine

A project-based guide to key-value storage, LSM-tree architecture, persistence, recovery, compaction, and database interfaces.

Implementation: C++17 storage engine and HTTP backend

Frontend: React + TypeScript

Build: CMake Testing: GoogleTest

Project repository: github.com/Amrittt22/StrataDB

Author: Amritanshu Singh

Preface — About This Book

StrataDB is a key-value storage engine built from scratch in C++17. Its architecture is inspired by Log-Structured Merge Trees (LSM-Trees), a family of designs used by modern storage systems.

This textbook follows data through the system: from an API request, to the Write-Ahead Log (WAL), to the in-memory MemTable, to immutable SSTables, and finally through reads, recovery, and compaction. It also covers the HTTP API, dashboard, testing, benchmarking, and the boundaries of the current implementation.

Accuracy note: General database concepts are distinguished from StrataDB-specific behavior. Where the current implementation does not provide a production-grade guarantee, the limitation is stated directly. The repository source code remains the authority for exact implementation details.

How to use this book

• Read Parts I and II in order if you are new to database internals.

• Read the component chapters alongside the C++ source code if you want to understand implementation.

• Use Part IV to build, run, test, and benchmark the project.

• Read Part V to understand what the project demonstrates and where its current engineering boundaries are.

Table of Contents

• Part I — Database Fundamentals: Storage Engines; LSM-Trees; Sequence Numbers and Tombstones.

• Part II — StrataDB Storage Engine: MemTable; Skip Lists; WAL; SSTables; Bloom Filters; Reads and Range Scans; Compaction; Manifest and Recovery.

• Part III — Application Layer: C++ REST API; React and TypeScript Dashboard.

• Part IV — Build, Test, and Measure: Build Instructions; Testing; Benchmarking.

• Part V — Design Boundaries: What StrataDB Demonstrates; Limitations; Source-Code Reading Guide; Glossary.

PART I — DATABASE FUNDAMENTALS

Chapter 1 — Understanding Storage Engines

A storage engine is the component of a database responsible for storing, retrieving, updating, and deleting data. An application may expose SQL, an HTTP API, or another interface. Beneath that interface, the storage engine manages how information is represented in memory and persisted to disk.

1.1 Key-value databases

A key-value database stores pairs in which a key identifies an associated value.

```
"name"       -> "Amritanshu"
"city"       -> "Bengaluru"
"language"   -> "C++"
"project"    -> "StrataDB"
```

A basic key-value engine needs to support Put, Get, Delete, and often Scan operations.

1.2 Memory and persistence

Location	Characteristics
RAM	Fast access, temporary storage
Disk	Persistent storage, different access costs

An in-memory map loses its contents when the process terminates. A persistent storage engine must preserve information across restarts and coordinate memory structures with files on disk.

1.3 Why database engineering is difficult

• Efficient reads and writes.

• Correct updates and deletion semantics.

• Recovery after unexpected termination.

• Multiple versions of a key.

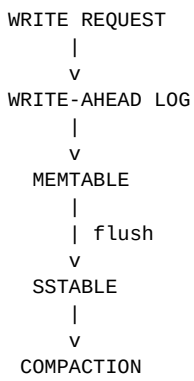
• Storage-file organization and metadata consistency.

• Efficient ordered range queries.

StrataDB explores these challenges through an LSM-Tree-inspired design.

Chapter 2 — Understanding LSM-Trees

An LSM-Tree, or Log-Structured Merge Tree, is a family of storage designs that organize writes in stages. Changes are initially recorded in memory and periodically written into sorted immutable files.



2.1 Why use this architecture?

Sequential log appends and batched flushes can reduce the need for random disk updates. This is one reason LSM-based designs are useful for write-heavy workloads.

2.2 The read trade-off

A key might exist in the MemTable or in several SSTables. A read may need to consult multiple locations and determine which version is newest. Bloom Filters and compaction help control this work.

2.3 Compaction

Compaction merges SSTables into a more consolidated representation. It can merge sorted records, resolve multiple versions of a key, and reduce redundant data. It consumes CPU and disk I/O and must be coordinated carefully with readers and crash recovery.

Chapter 3 — Sequence Numbers and Tombstones

A key-value database must handle updates correctly. If a key is written more than once, the engine needs a reliable way to identify the current version.

```
sequence 1: color -> blue
sequence 2: color -> green
sequence 3: color -> red
```

StrataDB uses sequence numbers to order operations. Later operations receive higher sequence numbers, and the newest applicable version wins.

3.1 Tombstones

Simply removing a key from the current MemTable is insufficient if an older SSTable still contains its value. A tombstone is a special versioned record representing deletion.

```
sequence 4: user:101 -> Amritanshu
sequence 9: user:101 -> TOMBSTONE
```

If the newest version is a tombstone, the key is considered absent. This rule must be respected during reads, flushing, recovery, and compaction.

PART II — STRATADB STORAGE ENGINE

Chapter 4 — MemTable

The MemTable is the in-memory component responsible for holding recent database operations. StrataDB uses a Skip List to maintain records in sorted key order.

4.1 Responsibilities

- Stores recent key-value records.
- Maintains ordered keys.
- Tracks sequence numbers.
- Represents deletions using tombstones.
- Supports lookup and ordered traversal.
- Provides data for SSTable flushing.
- Participates in threshold-based automatic flush.

4.2 Automatic flushing

When the configured MemTable threshold is reached, its records are flushed into an SSTable.

```
PUT key1
PUT key2
PUT key3
PUT key4
PUT key5
      |
      v
Threshold reached
      |
      v
```

```
MemTable flush
      |
      v
New SSTable
```

The threshold is configurable through the backend server configuration.

Chapter 5 — Skip Lists

A Skip List is a probabilistic ordered data structure. Its bottom level contains every element in sorted order, while higher levels contain shortcuts over portions of the bottom level.

```
Level 2: A -------- D -------- G
Level 1: A ---- C -- D ---- F -- G
Level 0: A -> B -> C -> D -> E -> F -> G
```

A search moves forward while the next key is smaller than the target, then drops to a lower level when necessary.

Operation	Expected complexity
Search	O(log n)
Insert	O(log n)
Delete	O(log n)

These are expected complexities under the usual randomized-level assumptions; worst-case behavior can be linear. A Skip List is useful in a MemTable because it supports ordered lookup, insertion, and traversal.

Chapter 6 — Write-Ahead Logging

A Write-Ahead Log is an append-oriented record of database operations. The central rule is that the log record is written before the corresponding operation is treated as represented in the in-memory database state.

6.1 Why a WAL is needed

Without a WAL, a process could accept a write into memory and lose it if the process exits before the MemTable is flushed. A WAL allows startup to replay recent operations.

6.2 StrataDB WAL V2

Field	Meaning
Operation	Put or delete
Sequence number	Operation version
Key length	Number of key bytes
Value length	Number of value bytes
Key / Value	Variable-length record data

6.3 Replay and checkpoints

During recovery, the engine reads WAL records and reapplies operations newer than the checkpoint represented by the Manifest.

```
for each valid WAL record:
    if record.sequence > checkpoint_sequence:
        replay(record)
```

StrataDB includes regression coverage for truncated WAL payload handling. This is not a guarantee against every possible disk corruption or hardware failure.

6.4 Durability boundary

Writing data through a file API does not automatically mean it has reached stable physical storage. StrataDB does not currently claim a complete explicit durability strategy using operating-system flush primitives such as fsync or Windows FlushFileBuffers.

Chapter 7 — SSTables

An SSTable, or Sorted String Table, is an immutable file containing records ordered by key. New changes are written to the MemTable and later become new SSTables.

7.1 StrataDB SSTable V3

The documented V3 format uses the magic identifier:

```
STRATAS3
```

Its records carry key/value information, tombstone state, and sequence numbers. These fields allow the engine to compare versions across files. The exact binary layout is defined by the SSTable implementation; this chapter describes the logical contents rather than promising a stable external file-format specification.

7.2 Flush lifecycle

1. Obtain an ordered view of the MemTable.

2. Write the records to a new SSTable.

3. Register the resulting file with the SSTable management layer.

4. Advance checkpoint metadata as implemented by the database.

5. Retire the flushed in-memory/WAL state according to the checkpoint process.

The ordering matters. A crash at the wrong point can leave metadata and files inconsistent.

Chapter 8 — Bloom Filters

A Bloom Filter is a space-efficient probabilistic structure used to test whether an item may be in a set.

• Definitely not present.

• Possibly present.

A Bloom Filter can produce false positives, but a correctly constructed filter does not produce false negatives for inserted items. For an SSTable lookup, a negative result allows the engine to skip that file. A positive result means the key may be present, so the actual lookup is still required.

A Bloom Filter does not store values and does not prove that a key exists. It helps avoid unnecessary searches.

Chapter 9 — Reads and Range Scans

9.1 Point lookup

1. Check the MemTable.

2. Examine candidate SSTables if the key is not resolved.

3. Use Bloom Filters to skip files that definitely cannot contain the key.

4. Find matching records in candidate files.

5. Compare sequence numbers.

6. Return the newest value, or absence if the newest version is a tombstone.

The newest version must win even if it is stored in a different SSTable. File creation order alone is not a sufficient substitute for version comparison.

9.2 Range scan

A range scan retrieves visible records in an ordered interval.

```
start = "apple"
end   = "orange"
```

The engine must produce ordered keys within the requested boundaries and resolve duplicate versions across storage locations. Tombstoned keys must not appear as live results.

Chapter 10 — SSTable Manager and Compaction

The SSTable Manager coordinates the collection of immutable files. It tracks available SSTables, searches them, resolves versions, and performs compaction.

10.1 Compaction example

```
SSTable A:
a -> old       sequence 2
b -> value-b   sequence 3

SSTable B:
a -> new       sequence 7
b -> TOMBSTONE sequence 8
c -> value-c   sequence 9
```

After merging, the newest version for each key is selected:

```
a -> new       sequence 7
b -> TOMBSTONE sequence 8
c -> value-c   sequence 9
```

The old version of a is obsolete. The tombstone for b must not be discarded while an older value might still be found in another file.

StrataDB's current compaction behavior retains tombstones. This avoids accidentally revealing an older value after a deletion. Crash-safe compaction and orphan-file recovery remain limited.

Chapter 11 — Manifest and Checkpoint Recovery

The Manifest records checkpoint metadata used during recovery. A checkpoint sequence number identifies operations already represented by checkpointed storage.

1. Initialize database metadata.
2. Read the Manifest checkpoint.
3. Load the available SSTable state.
4. Read WAL records.
5. Ignore records covered by the checkpoint.
6. Replay newer valid records into the MemTable.
7. Make the recovered database available.

Recovery must avoid losing writes promised by its durability contract, replaying stale operations, resurrecting deleted keys, or treating incomplete records as valid operations.

StrataDB includes tests for checkpoint-aware replay, persistence across restart, and truncated WAL payload handling. These tests cover specific scenarios, not every possible crash or storage-device failure.

PART III — APPLICATION LAYER

Chapter 12 — C++ HTTP REST API

The storage engine is exposed through a C++ HTTP server using cpp-httplib. The server provides a network boundary between the database and client applications.

Method	Endpoint	Purpose
GET	/health	Health check
PUT	/kv/:key	Insert or update a key
GET	/kv/:key	Retrieve a key
DELETE	/kv/:key	Delete a key
GET	/scan?start=a&end;=z	Scan an ordered key range
GET	/stats	Read storage statistics

12.1 Example requests

```
PUT /kv/language
Content-Type: text/plain

C++

GET /kv/language

DELETE /kv/language

GET /scan?start=a&end=z

GET /stats
```

12.2 Server configuration

Setting	Default
Host	127.0.0.1
Port	8080
Data directory	./stratadb_data
MemTable flush threshold	1000

The server accepts command-line configuration for the data directory, port, and flush threshold. The current server serializes database requests using a mutex. This protects shared state in this implementation but limits parallel request execution.

The development CORS configuration and absence of authentication/TLS mean the server should not be exposed as a production service without additional security work.

Chapter 13 — React and TypeScript Dashboard

The frontend provides a visual interface for interacting with the database.

- Backend health and connection status.
- Insert and update key-value records.
- Retrieve and delete records.
- Scan a key range.
- View MemTable and SSTable statistics.
- Display automatic flush status.
- Refresh statistics.

The frontend API integration is located at:

```
frontend/src/api/stratadb.ts
```

The backend URL is configured through:

```
VITE_API_BASE_URL=http://localhost:8080
```

The dashboard is a client of the database, not part of the storage engine itself. Other HTTP clients can use the same API.

PART IV — BUILD, TEST, AND MEASURE

Chapter 14 — Building and Running StrataDB

14.1 Prerequisites

• C++17 compiler.

• CMake and Ninja.

• Git.

• Node.js and npm for the frontend.

The documented Windows development environment uses MSYS2 UCRT64 GCC.

14.2 Clone the repository

```
git clone https://github.com/Amrittt22/StrataDB.git
cd StrataDB
```

14.3 Configure CMake

```
$env:PATH = "C:\\msys64\\ucrt64\\bin;$env:PATH"

cmake -S . -B build -G Ninja `
  -DCMAKE_CXX_COMPILER="C:/msys64/ucrt64/bin/g++.exe"
```

14.4 Build and run backend

```
cmake --build build

.\build\stratadb_server.exe
```

With custom settings:

```
.\build\stratadb_server.exe .\my_database 8080 1000
```

14.5 Run frontend

```
cd frontend
npm install
```

Create a local .env file based on .env.example:

```
VITE_API_BASE_URL=http://localhost:8080
```

Then start the frontend:

```
npm run dev
```

Open the local URL printed by Vite, normally http://localhost:5173. Keep the C++ backend and React development server running simultaneously.

Chapter 15 — Testing the Engine

Testing matters especially for storage software because bugs may appear only after sequences of writes, deletes, flushes, compactions, and restarts.

The GoogleTest suite covers core components such as Bloom Filter, database operations, logger, MemTable, Skip List, SSTable, SSTable Manager, statistics, and WAL.

Regression scenarios include sequence-aware recovery, checkpoint handling, automatic flush, truncated WAL payload handling, compaction with multiple SSTables, duplicate keys and newer versions, tombstone behavior, and restart-related storage behavior.

15.1 Run the suite

```
ctest --test-dir build --output-on-failure
```

Latest recorded development result: 109/109 tests passing. Run the command against your own checkout and environment rather than assuming a previous result guarantees your build.

A passing test suite does not prove the absence of all bugs. Storage engines also need crash injection, filesystem fault testing, concurrency stress tests, and durability testing to establish stronger guarantees.

15.2 Integration verification

- Health checks.
- PUT and update operations.
- GET operations.
- Range scans.
- Statistics.
- DELETE operations and missing-key responses.
- Automatic flushing through the configured threshold.
- Restart persistence for the tested scenario.

Chapter 16 — Benchmarking

StrataDB includes a standalone benchmark executable.

Operation	What it measures
PUT	Writes into the MemTable
GET	Reads from a database prepared with flushed SSTable data
Range scan	Ordered retrieval over a key interval
Compaction	Time taken by compaction

The CSV output includes dataset size, operation count, total time, throughput, average latency, p50/p95/p99 latency, and returned record count where applicable.

16.1 Build and run

```
cmake --build build
.\build\stratadb_benchmark.exe
```

Specify a dataset size:

```
.\build\stratadb_benchmark.exe 5000
```

Specify a dataset size and output file:

```
.\build\stratadb_benchmark.exe 1000 .\benchmark-results.csv
```

The default output location is:

```
benchmarks/results/stratadb_benchmark.csv
```

16.2 Initial local results

These are initial local development measurements, not production performance claims.

Operation	500 records	1,000 records	5,000 records
PUT throughput (ops/sec)	2,011.74	1,230.51	346.69
GET throughput (ops/sec)	399,265.35	1,120,071.69	448,153.16
Range scan time (ms, 3 scans)	5.936	9.851	27.221
Compaction time (ms)	16.632	23.436	50.198
Verified records after compaction	500	1,000	5,000

Interpretation requires care: results depend on hardware, compiler, operating system, and runtime conditions. GET measurements may benefit from operating-system caching. Range scans use only three repetitions per dataset, too few for strong percentile conclusions. Compare engines only when datasets, durability settings, workloads, and environments are comparable.

PART V — DESIGN BOUNDARIES

Chapter 17 — What StrataDB Demonstrates

1. Data structures: Skip Lists and Bloom Filters.

2. Persistence: WAL records and immutable SSTable files.

3. Versioning: sequence numbers and newest-version resolution.

4. Deletion semantics: tombstones.

5. Organization: SSTable management and compaction.

6. Recovery: Manifest checkpoints and WAL replay.

7. Interfaces: C++ REST API and React dashboard.

8. Engineering practice: unit tests, regression tests, integration verification, and benchmarks.

The key learning outcome is understanding how these pieces depend on one another. A tombstone affects reads, compaction, and recovery. A checkpoint's relationship to SSTables and the WAL determines what can safely be replayed.

Chapter 18 — Current Limitations and Production Considerations

StrataDB is an educational and portfolio-oriented storage engine. It should not yet be treated as a production database.

• No complete explicit operating-system-level durability strategy using fsync or Windows FlushFileBuffers.

• Limited crash consistency for compaction and orphan-file recovery.

• Need for broader crash and recovery testing.

- Need for stronger Manifest/SSTable consistency validation.
- Mutex-based request serialization rather than a highly concurrent execution model.
- No authentication or TLS.
- Development-oriented CORS configuration.
- Need for broader benchmark comparisons and workload coverage.

These are important boundaries. Implementing a database concept is not the same as establishing production-grade guarantees.

Chapter 19 — How to Explore the Source Code

1. Record and data model: understand what a stored record contains.
2. Skip List and MemTable: understand in-memory ordering and writes.
3. WAL: understand operation serialization and replay.
4. SSTable: understand the persistent sorted-file representation.
5. Bloom Filter: understand lookup filtering.
6. SSTable Manager: understand multi-file lookup and compaction.
7. Manifest: understand checkpoint metadata.
8. Database: understand how components are coordinated.
9. HTTP server: understand how database operations are exposed.
10. Frontend API client and UI: understand how the dashboard consumes the service.
11. Tests and benchmark: understand which behaviors are verified and how performance is measured.

Useful top-level paths:

```
engine/include/
engine/src/
engine/tests/
engine/benchmarks/
frontend/src/
CMakeLists.txt
```

File names and exact implementation details should be confirmed against the checked-out repository. The source code is the authority for implementation behavior.

Glossary

Term	Meaning
Key-value store	A database organized around keys and associated values
Storage engine	Subsystem responsible for data storage and retrieval
LSM-Tree	A design that stages writes in memory and merges sorted disk files
MemTable	In-memory structure holding recent database operations
Skip List	Probabilistic ordered data structure
WAL	Write-Ahead Log; a record of operations used for recovery
SSTable	Immutable sorted file containing records
Bloom Filter	Probabilistic structure that can rule out membership
Sequence number	Ordered version identifier for an operation
Tombstone	Deletion marker retained so older values do not reappear
Flush	Writing MemTable contents into persistent sorted storage
Compaction	Merging SSTables and resolving obsolete versions
Manifest	Metadata that tracks checkpoint/storage state
Checkpoint	Boundary indicating operations already represented in persistent state
Recovery	Reconstructing usable database state after restart
Range scan	Retrieving records over an ordered key interval
Throughput	Number of operations completed per unit of time
Latency	Time taken by an operation
p95 latency	Latency threshold at or below which 95% of measured operations fall, subject to the percentile method

Final Notes

StrataDB is a hands-on exploration of the internal mechanics of a database storage engine. Its value is not only in the final interface, but in the relationships between the WAL, MemTable, SSTables, Bloom Filters, sequence numbers, tombstones, compaction, and recovery.

Use this book alongside the source code. Trace one write through the system, follow it into an SSTable, restart the database, and observe how reads and compaction resolve versions.

The best way to learn a storage engine is to follow the data.

Project repository: https://github.com/Amrittt22/StrataDB
Author: Amritanshu Singh